

Конспект: Tarkov Kappa Tracker

Личный сайт-трекер для прохождения квестов Escape From Tarkov до контейнера Карра.

Проект построен с нуля без опыта в веб-разработке. Конспект охватывает каждую строчку кода, объясняет, зачем она нужна, и даёт ссылки для углублённого изучения.

Оглавление

1. [Что мы построили](#)
 2. [Архитектура проекта](#)
 3. [Используемые библиотеки и инструменты](#)
 4. [Структура проекта](#)
 5. [Подробный разбор файлов](#)
 6. [Базовые концепции, которые ты освоил](#)
 7. [Что делать, когда что-то идёт не так](#)
 8. [Как развивать проект дальше](#)
-

Что мы построили

Локальный веб-сайт, который работает только на твоём компьютере (`http://localhost:8000`). Он:

- Загружает ~500 квестов из публичного API `tarkov.dev`
- Строит граф зависимостей квестов (какой квест нужен для какого)
- Хранит твой личный прогресс в JSON-файле
- Показывает список доступных прямо сейчас квестов с учётом твоей фракции (BEAR/USEC), уровня и выполненных квестов
- Позволяет отмечать квесты выполненными в один клик

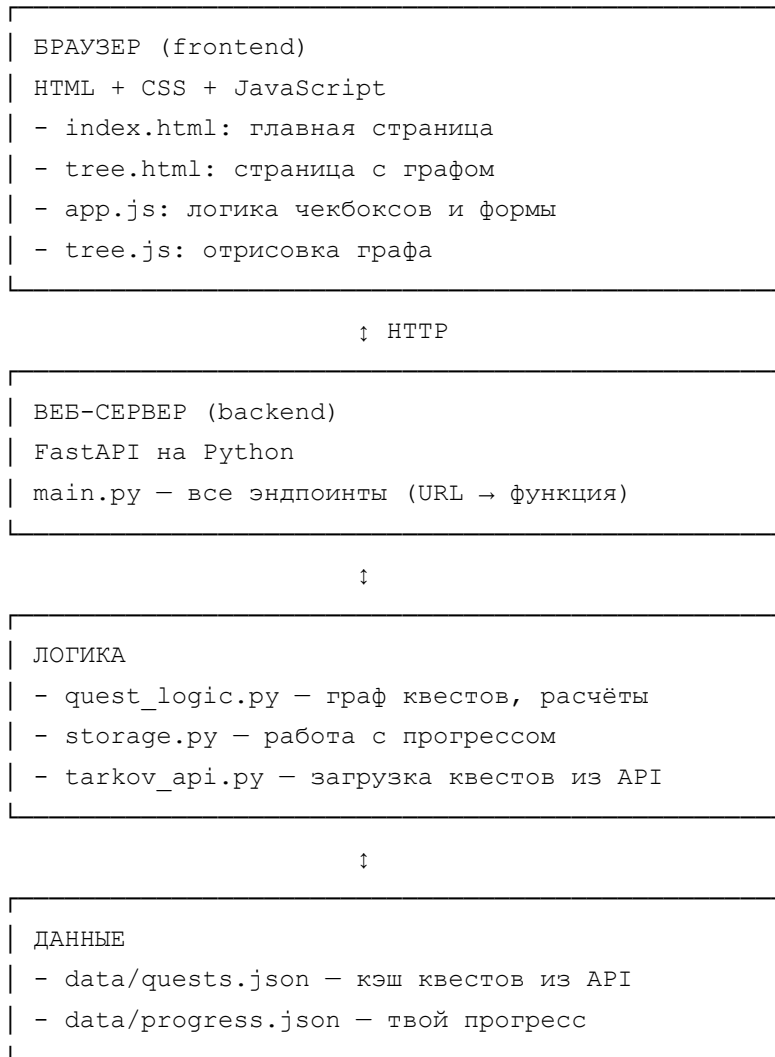
- Визуализирует дерево квестов до Карра с подсветкой статусов

Стек технологий:

- Backend: Python 3.10+, FastAPI, networkx, httpx
- Frontend: HTML, CSS, vanilla JavaScript, Cytoscape.js
- Хранилище: JSON-файлы
- Источник данных: tarkov.dev GraphQL API

Архитектура проекта

Проект разделён на чёткие слои — это называется **разделение ответственности** (separation of concerns). Каждый файл отвечает за свою задачу, и его можно менять независимо от других.



Принцип: браузер не знает про логику квестов — он просто отправляет запросы и отображает данные. Сервер не знает, как рисуется граф — он просто отдаёт данные. Логика квестов не знает про веб — она просто отвечает на вопросы про граф.

Этот подход называется **многослойная архитектура**. Каждый слой делает своё дело и общается только со своими соседями.

Используемые библиотеки и инструменты

Python (backend)

Python 3.10+

Сам язык программирования. Версия 3.10+ нужна для современного синтаксиса типов (`list[dict]` вместо `List[Dict]`).

- Документация: <https://docs.python.org/3/>
- Тьюториал на русском: <https://pythontutor.ru/>

venv (виртуальное окружение)

Встроенный модуль Python. Создаёт изолированную папку с библиотеками только для этого проекта. Без него библиотеки разных проектов конфликтуют.

- Документация: <https://docs.python.org/3/library/venv.html>

pip

Менеджер пакетов Python. Устанавливает библиотеки из общего репозитория PyPI.

- Документация: <https://pip.pypa.io/>

httpx

Современная библиотека для HTTP-запросов. Аналог легендарного `requests` , но умеет асинхронный режим.

- Документация: <https://www.python-httpx.org/>
- Зачем нам: ходить в API tarkov.dev

networkx

Библиотека для работы с графами. Узлы, рёбра, обходы, поиски пути, топологическая сортировка.

- Документация: <https://networkx.org/>
- Тьюториал: <https://networkx.org/documentation/stable/tutorial.html>
- Зачем нам: построить граф зависимостей квестов и находить пути в нём

FastAPI

Современный веб-фреймворк для Python. Быстрый, с автоматической валидацией данных и автогенерацией документации.

- Документация: <https://fastapi.tiangolo.com/>
- Тьюториал на русском (официальный): <https://fastapi.tiangolo.com/ru/tutorial/>
- Зачем нам: принимать HTTP-запросы и отдавать страницы или JSON

Pydantic

Идёт в комплекте с FastAPI. Описывает структуры данных и автоматически валидирует входящие запросы.

- Документация: <https://docs.pydantic.dev/>
- Зачем нам: проверять, что в POST-запросах приходят правильные поля правильных типов

uvicorn

ASGI-сервер. Запускает наше FastAPI-приложение и слушает HTTP-запросы.

- Документация: <https://www.uvicorn.org/>
- Зачем нам: чтобы FastAPI начал реально работать

Jinja2

Шаблонизатор HTML. Позволяет вставлять данные Python в HTML-страницы.

- Документация: <https://jinja.palletsprojects.com/>
- Зачем нам: отдавать HTML с подставленными данными (имя пользователя, список квестов и т.д.)

Frontend

HTML

Язык разметки веб-страниц. Описывает структуру: заголовки, абзацы, кнопки, формы.

- Тьюториал: <https://developer.mozilla.org/ru/docs/Learn/HTML>

CSS

Язык стилей. Описывает, как страница выглядит: цвета, шрифты, расположение.

- Тьюториал: <https://developer.mozilla.org/ru/docs/Learn/CSS>

JavaScript

Язык программирования для браузера. Делает страницы интерактивными.

- Тьюториал: <https://learn.javascript.ru/> — лучший русскоязычный учебник
- Документация: <https://developer.mozilla.org/ru/docs/Web/JavaScript>

Cytoscape.js

JavaScript-библиотека для визуализации графов в браузере.

- Документация: <https://js.cytoscape.org/>
- Зачем нам: нарисовать дерево квестов

dagre

Алгоритм иерархической раскладки графов. Подключается как плагин к Cytoscape.

- Документация: <https://github.com/dagrejs/dagre/wiki>

Внешние сервисы

tarkov.dev API

Публичное GraphQL-API с актуальными данными игры.

- Главная: <https://tarkov.dev/>
- Документация API: <https://api.tarkov.dev/>

GraphQL

Стандарт для API, при котором клиент сам описывает, какие поля ему нужны. Альтернатива классическому REST.

- Документация: <https://graphql.org/learn/>
- Зачем нам: одним запросом получить от tarkov.dev все нужные поля квестов

Инструменты разработки

VS Code

Бесплатный редактор кода от Microsoft. Подсветка синтаксиса, автодополнение, встроенный терминал.

- Скачать: <https://code.visualstudio.com/>

PowerShell

Командная оболочка Windows. Через неё запускаем Python-команды.

Браузер DevTools

Встроенный инструмент любого браузера (открыть: F12). Главные вкладки: Console (логи и ошибки), Network (HTTP-запросы), Elements (структура страницы).

- Подробно: <https://developer.chrome.com/docs/devtools/>

Структура проекта

```
tarkov_kappa/
├── venv/                # Виртуальное окружение (не трогаем)
├── app/                 # Весь backend-код
│   ├── __pycache__/    # Кэш Python (создается автоматически)
│   ├── templates/      # HTML-шаблоны Jinja2
│   │   ├── base.html   # Общий каркас всех страниц
│   │   ├── index.html  # Главная страница
│   │   └── tree.html    # Страница с графом
│   ├── tarkov_api.py   # Загрузка квестов из tarkov.dev
│   ├── quest_logic.py  # Граф зависимостей и логика
│   ├── storage.py      # Работа с прогрессом
│   └── main.py          # Веб-сервер FastAPI
```

```
├─ static/                # Файлы для браузера (CSS, JS)
│   ├── style.css         # Стили
│   ├── app.js            # Скрипт главной страницы
│   └─ tree.js            # Скрипт страницы с графом
├─ data/                  # Данные
│   ├── quests.json       # Кэш квестов
│   └─ progress.json      # Твой прогресс
└─ requirements.txt        # Список Python-библиотек
```

Подробный разбор файлов

`app/tarkov_api.py` — загрузка данных из API

Этот модуль один раз загружает квесты с `tarkov.dev` и сохраняет их в локальный JSON. Запускается отдельно от сервера, при необходимости обновить данные.

```
import json
from pathlib import Path

import httpx
```

Импорты:

- `json` — встроенный модуль для работы с JSON. Превращает Python-объекты в JSON-строки и обратно.
- `Path` из `pathlib` — современный способ работать с путями к файлам. Кроссплатформенный (работает одинаково на Windows и Linux).
- `httpx` — наша библиотека для HTTP-запросов.

```
API_URL = "https://api.tarkov.dev/graphql"
```

URL единого GraphQL-эндпоинта `tarkov.dev`. У GraphQL всегда **один URL**, в отличие от REST, где их много (`/quests` , `/items` , и т.д.).

```
QUESTS_QUERY = """
{
  tasks(lang: ru) {
    id
    name
    minPlayerLevel
```

```

        kappaRequired
        factionName
        trader { name }
        map { name }
        taskRequirements {
            task { id name }
        }
        objectives { description }
    }
}
"""

```

GraphQL-запрос. Описывает, какие данные мы хотим получить. Сервер вернёт ровно эти поля, ничего лишнего.

- `tasks(lang: ru)` — все квесты, названия на русском
- `id, name` — основные идентификаторы
- `minPlayerLevel` — минимальный уровень игрока
- `kappaRequired` — флаг «нужен для Карра»
- `factionName` — для какой фракции (USEC/BEAR/Any)
- `trader { name }` — у GraphQL для вложенных объектов нужно явно указать поля
- `taskRequirements` — список квестов-предшественников. Это сердце графа зависимостей.
- `objectives` — задачи внутри квеста (что конкретно нужно сделать)

```

DATA_DIR = Path(__file__).resolve().parent.parent / "data"
QUESTS_FILE = DATA_DIR / "quests.json"

```

Вычисляем абсолютные пути:

- `__file__` — путь к текущему файлу `tarkov_api.py`
- `.resolve()` — превращает в абсолютный (полный) путь
- `.parent` — папка, где лежит файл (`app/`)
- `.parent.parent` — на уровень выше (`tarkov_kappa/`)
- `/ "data"` — оператор `/` у `Path` добавляет подпапку (умно, читается как путь)

Зачем так сложно? Чтобы скрипт работал из любой папки. Если бы написали просто

"data/quests.json" , то при запуске не из корня проекта файл не нашёлся бы.

```
def fetch_quests() -> list[dict]:
    print("Отправляю запрос на tarkov.dev ...")
    response = httpx.post(
        API_URL,
        json={"query": QUESTS_QUERY},
        timeout=30.0,
    )
    response.raise_for_status()
    data = response.json()
    if "errors" in data:
        raise RuntimeError(f"GraphQL вернул ошибки: {data['errors']}")
    quests = data["data"]["tasks"]
    print(f"Получено квестов: {len(quests)}")
    return quests
```

Функция `fetch_quests` — отправляет запрос и возвращает список квестов.

- `httpx.post(API_URL, json={"query": ...}, timeout=30.0)` — POST-запрос с JSON-телом. Ждём максимум 30 секунд.
- `response.raise_for_status()` — если сервер вернул HTTP-ошибку (4xx или 5xx), бросаем исключение. Без этого мы бы получили пустые данные и не поняли, в чём дело.
- `response.json()` — превращаем тело ответа в Python-словарь.
- Проверка `"errors" in data` — особенность GraphQL: ошибки возвращаются не HTTP-кодом, а в теле ответа. Поэтому проверяем явно.
- `data["data"]["tasks"]` — стандартная структура GraphQL-ответа: всё лежит под ключом `data`.

Тип-хинт `-> list[dict]` — подсказка редактору и тебе, что функция возвращает список словарей. На выполнение не влияет, но помогает писать корректный код.

```
def save_quests(quests: list[dict]) -> None:
    DATA_DIR.mkdir(exist_ok=True)
    with open(QUESTS_FILE, "w", encoding="utf-8") as f:
        json.dump(quests, f, ensure_ascii=False, indent=2)
    print(f"Сохранено в {QUESTS_FILE}")
```

Сохранение в файл.

- `DATA_DIR.mkdir(exist_ok=True)` — создаём папку, если её нет. `exist_ok=True` означает «не падай, если уже есть».
- `with open(..., "w", encoding="utf-8") as f:` — открываем файл для записи в UTF-8. Конструкция `with` гарантирует закрытие файла, даже если случится ошибка.
- `json.dump(quests, f, ensure_ascii=False, indent=2)` — пишем словарь как JSON. `ensure_ascii=False` нужно для русского текста (иначе кириллица превратится в `\u0xxx`). `indent=2` — красивое форматирование.

```
if __name__ == "__main__":
    main()
```

Идиома Python. Срабатывает только если файл запустили напрямую (`python -m app.tarkov_api`). Если же его импортируют как модуль из другого файла — не срабатывает. Полезно для модулей, которые могут работать и как библиотека, и как самостоятельный скрипт.

Запуск: `python -m app.tarkov_api`

`app/quest_logic.py` — граф квестов

Самый «умный» модуль. Строит граф зависимостей и отвечает на сложные вопросы.

Что такое граф

Граф — это структура из **узлов** (nodes) и **рёбер** (edges) между ними. Например, метро: станции — узлы, тоннели — рёбра.

Наш граф:

- Узлы — квесты
- Рёбра — зависимости («квест A нужен перед квестом B» = ребро $A \rightarrow B$)
- Граф **направленный** (стрелочки имеют направление) — DiGraph
- Граф **ациклический** (нет циклов $A \rightarrow B \rightarrow A$) — DAG (Directed Acyclic Graph)

DAG — стандартная штука в программировании. Используется в системах сборки (Make, Webpack), планировщиках задач, базах данных.

```
import json
```

```
from pathlib import Path
import networkx as nx
```

`networkx` принято импортировать как `nx` — короче и общепринято.

```
def build_graph(quests: list[dict]) -> nx.DiGraph:
    graph = nx.DiGraph()

    # Сначала добавляем все квесты как узлы с их данными
    for quest in quests:
        graph.add_node(
            quest["id"],
            name=quest["name"],
            trader=quest["trader"]["name"] if quest.get("trader") else "Неизвестн
            map=quest["map"]["name"] if quest.get("map") else None,
            min_level=quest.get("minPlayerLevel", 0),
            kappa_required=quest.get("kappaRequired", False),
            faction=quest.get("factionName", "Any"),
            objectives=[obj["description"] for obj in quest.get("objectives", [])
        )

    # Потом добавляем рёбра на основе taskRequirements
    for quest in quests:
        for requirement in quest.get("taskRequirements", []):
            required_task = requirement.get("task")
            if required_task:
                graph.add_edge(required_task["id"], quest["id"])

    return graph
```

Построение графа. Делаем в **два прохода** — сначала все узлы, потом все рёбра. Иначе при добавлении ребра второй узел может ещё не существовать, и `networkx` создаст «узел-призрак» без данных.

`graph.add_node(id, name=..., trader=...)` — `networkx` позволяет прикреплять любые данные к узлам. Потом доступ через `graph.nodes[id]["name"]`. Это превращает граф в полноценное хранилище.

`quest["trader"]["name"] if quest.get("trader") else "Неизвестно"` — защита от `None`. У некоторых квестов нет торговца, и без проверки код упадёт с ошибкой `'NoneType' has no attribute 'name'`. Такие проверки — суть работы с реальными данными.

`graph.add_edge(required_task["id"], quest["id"])` — направление ребра: «от требуемого квеста к текущему». То есть ребро показывает «открывает следующий

КВЕСТ».

```
def get_kappa_quests(graph: nx.DiGraph) -> set[str]:
    direct_kappa = {
        node for node, data in graph.nodes(data=True)
        if data.get("kappa_required")
    }
    all_kappa = set(direct_kappa)
    for quest_id in direct_kappa:
        all_kappa.update(nx.ancestors(graph, quest_id))
    return all_kappa
```

Получение всех Карра-квестов.

- Сначала находим квесты с явным флагом `kappaRequired=True`
- Потом для каждого добавляем **всех предков** через `nx.ancestors`. Это транзитивный обход вверх по графу: предки + предки предков + ...

`{node for node, data in ...}` — это **set comprehension** (генератор множества). Аналогично `list comprehension` `[x for x in ...]`, но создаёт `set` вместо `list`.

Используем `set` (множество), а не `list`, потому что:

- Множества автоматически удаляют дубликаты
- Проверка `x in s` для множества работает за $O(1)$, а для списка — $O(n)$

```
def get_available_quests(
    graph: nx.DiGraph,
    completed: set[str],
    faction: str = "BEAR",
    player_level: int = 1,
    kappa_only: bool = True,
) -> list[dict]:
    candidates = get_kappa_quests(graph) if kappa_only else set(graph.nodes)
    available = []

    for quest_id in candidates:
        if quest_id in completed:
            continue

        data = graph.nodes[quest_id]

        quest_faction = data.get("faction", "Any")
        if quest_faction not in ("Any", faction):
```

```

        continue

    if data["min_level"] > player_level:
        continue

    predecessors = set(graph.predecessors(quest_id))
    if not predecessors.issubset(completed):
        continue

    available.append({
        "id": quest_id,
        "name": data["name"],
        "trader": data["trader"],
        "map": data["map"],
        "min_level": data["min_level"],
        "faction": quest_faction,
    })

    available.sort(key=lambda q: (q["trader"], q["name"]))
    return available

```

Доступные сейчас квесты. Самая важная функция модуля.

Параметры:

- `completed` — что уже выполнено (`set` для быстрых проверок)
- `faction` — фракция игрока (фильтрация)
- `player_level` — уровень (фильтрация)
- `kappa_only` — рассматривать только Карра-квесты или вообще все

Логика для каждого квеста:

1. Уже выполнен? Пропускаем.
2. Не подходит по фракции? Пропускаем.
3. Уровень игрока ниже требуемого? Пропускаем.
4. Не все предшественники выполнены? Пропускаем.
5. Иначе — добавляем в результат.

`predecessors.issubset(completed)` — проверка «множество предшественников $\subseteq$ множество выполненных». Если у квеста нет предшественников, `predecessors` пусто, а пустое множество — подмножество любого, поэтому такой квест автоматически

доступен (если фракция и уровень подходят).

`available.sort(key=lambda q: (q["trader"], q["name"]))` — сортировка по двум ключам: сначала по торговцу, потом по названию. `lambda` — анонимная функция в одну строку.

```
def get_quest_order(graph: nx.DiGraph, kappa_only: bool = True) -> list[str]:
    target_graph: nx.DiGraph = graph
    if kappa_only:
        kappa_ids = get_kappa_quests(graph)
        target_graph = graph.subgraph(kappa_ids)
    return list(nx.topological_sort(target_graph))
```

Топологическая сортировка — алгоритм для DAG, возвращает порядок узлов, при котором для каждого ребра $A \rightarrow B$ узел A идёт раньше B . То есть «корректный порядок выполнения задач с учётом зависимостей».

`graph.subgraph(kappa_ids)` — берём только часть графа с Карра-квестами.

Аннотация типа `target_graph: nx.DiGraph = graph` нужна, чтобы Pylance (анализатор типов в VS Code) не ругался на возвращаемый тип `subgraph()`.

app/storage.py — хранение прогресса

Простой модуль для работы с `progress.json`.

```
import json
from pathlib import Path
from typing import Literal

DATA_DIR = Path(__file__).resolve().parent.parent / "data"
PROGRESS_FILE = DATA_DIR / "progress.json"

Faction = Literal["USEC", "BEAR"]

DEFAULT_PROGRESS = {
    "faction": "USEC",
    "level": 1,
    "completed_quests": [],
}
```

`Literal["USEC", "BEAR"]` — тип, говорящий «может быть только одно из двух конкретных значений». Подсказка редактору.

```
def load_progress() -> dict:
    if not PROGRESS_FILE.exists():
        save_progress(DEFAULT_PROGRESS)
        return dict(DEFAULT_PROGRESS)

    with open(PROGRESS_FILE, "r", encoding="utf-8") as f:
        progress = json.load(f)

    for key, value in DEFAULT_PROGRESS.items():
        progress.setdefault(key, value)

    return progress
```

`dict(DEFAULT_PROGRESS)` — возвращаем **копию** словаря. Если бы вернули сам — изменения у вызывающего сломали бы наш «эталон». Классическая ловушка Python: словари и списки передаются по ссылке.

`progress.setdefault(key, value)` — добавляет ключ, только если его ещё нет. Защита на случай, если добавим новые поля в будущем.

```
def mark_completed(quest_id: str) -> dict:
    progress = load_progress()
    if quest_id not in progress["completed_quests"]:
        progress["completed_quests"].append(quest_id)
        save_progress(progress)
    return progress
```

Идемпотентность — повторный вызов с тем же `id` не создаст дубликата. Важное свойство для надёжных API.

app/main.py — веб-сервер

Точка входа в веб-приложение. Все эндпоинты живут здесь.

```
from pathlib import Path
from fastapi import FastAPI, Request, HTTPException
from fastapi.responses import HTMLResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel

from app import quest_logic, storage
```

`from app import quest_logic, storage` — импорт наших собственных модулей. Они импортируются один раз и доступны до конца работы сервера.

```
app = FastAPI(title="Tarkov Kappa Tracker")
```

```
templates = Jinja2Templates(directory=str(TEMPLATES_DIR))
```

```
app.mount("/static", StaticFiles(directory=str(STATIC_DIR)), name="static")
```

`app = FastAPI(...)` — создаём экземпляр приложения. К нему «прикрепляем» обработчики через декораторы.

`app.mount("/static", ...)` — говорим: «все запросы по адресу `/static/что-то` отдавай как файлы из папки `static/`». Так браузер получает CSS и JS.

```
print("Загружаю квесты и строю граф...")
QUESTS = quest_logic.load_quests()
GRAPH = quest_logic.build_graph(QUESTS)
KAPPA_IDS = quest_logic.get_kappa_quests(GRAPH)
```

Тяжёлая работа при старте. Загрузка квестов и построение графа происходят **один раз**, при запуске сервера. Результаты хранятся в памяти как глобальные переменные.

Альтернатива (плохая) — пересчитывать на каждый запрос. Тогда страницы открывались бы по полсекунды.

```
@app.get("/", response_class=HTMLResponse)
def index(request: Request):
    progress = storage.load_progress()
    completed = set(progress["completed_quests"])
    completed_kappa = completed & KAPPA_IDS
    available = quest_logic.get_available_quests(
        GRAPH,
        completed=completed,
        faction=progress["faction"],
        player_level=progress["level"],
    )

    context = {
        "request": request,
        "faction": progress["faction"],
        "level": progress["level"],
        "total_kappa": len(KAPPA_IDS),
        "completed_count": len(completed_kappa),
        "available": available,
    }
```



```
return templates.TemplateResponse(request, "index.html", context)
```

Эндпоинт главной страницы.

`@app.get("/")` — декоратор, регистрирующий функцию как обработчик GET-запросов на URL `/`.

`request: Request` — FastAPI смотрит на типы аргументов и автоматически подставляет нужные объекты. Это называется **Dependency Injection**.

`completed & KAPPA_IDS` — пересечение множеств. Берём только те выполненные квесты, которые входят в Карра-набор. Чтобы прогресс «47/257» считался правильно.

`templates.TemplateResponse(request, "index.html", context)` — рендерим HTML-шаблон с переданными данными.

```
class ToggleRequest(BaseModel):
    quest_id: str

@app.post("/api/progress/toggle")
def toggle_quest(payload: ToggleRequest):
    quest_id = payload.quest_id

    if quest_id not in GRAPH.nodes:
        raise HTTPException(status_code=404, detail=f"Квест {quest_id} не найден")

    progress = storage.load_progress()
    completed = set(progress["completed_questions"])

    if quest_id in completed:
        storage.mark_uncompleted(quest_id)
        new_status = "uncompleted"
    else:
        storage.mark_completed(quest_id)
        new_status = "completed"

    progress = storage.load_progress()
    completed = set(progress["completed_questions"])
    completed_kappa = completed & KAPPA_IDS
    available = quest_logic.get_available_questions(
        GRAPH,
        completed=completed,
        faction=progress["faction"],
        player_level=progress["level"],
    )
```

```

return {
    "status": new_status,
    "quest_id": quest_id,
    "completed_count": len(completed_kappa),
    "total_kappa": len(KAPPA_IDS),
    "available_ids": [q["id"] for q in available],
}

```

Эндпоинт переключения статуса квеста.

`class ToggleRequest(BaseModel)` — Pydantic-модель, описывающая структуру входящего JSON. FastAPI автоматически валидирует.

`@app.post(...)` — обработчик POST-запросов. POST используется для **изменения** данных (вспомни: GET читает, POST меняет).

`payload: ToggleRequest` — FastAPI разберёт тело запроса как JSON и провалидирует через Pydantic. Если придёт что-то не то — автоматически вернёт 422.

`raise HTTPException(status_code=404, ...)` — стандартный способ вернуть HTTP-ошибку с понятным текстом.

Возвращаемый словарь автоматически конвертируется FastAPI в JSON-ответ.

```

@app.get("/api/tree")
def get_tree():
    progress = storage.load_progress()
    completed = set(progress["completed_quests"])
    available_ids = {
        q["id"] for q in quest_logic.get_available_quests(
            GRAPH,
            completed=completed,
            faction=progress["faction"],
            player_level=progress["level"],
        )
    }

    nodes = []
    edges = []
    subgraph = GRAPH.subgraph(KAPPA_IDS)

    for node_id in subgraph.nodes:
        data = subgraph.nodes[node_id]
        if node_id in completed:
            status = "completed"

```

```

        elif node_id in available_ids:
            status = "available"
        else:
            status = "locked"

    nodes.append({
        "data": {
            "id": node_id,
            "label": data["name"],
            "trader": data["trader"],
            "status": status,
        }
    })

for source, target in subgraph.edges:
    edges.append({
        "data": {
            "id": f"{source}->{target}",
            "source": source,
            "target": target,
        }
    })

return {"nodes": nodes, "edges": edges}

```

Эндпоинт для графа. Возвращает данные в формате, который понимает Cytoscape.js:

{nodes: [...], edges: [...]}. Каждый узел и ребро обернуты в объект с ключом data .

Статус считаем здесь, на сервере. Фронту останется только отрисовать.

```

class SettingsRequest(BaseModel):
    faction: str
    level: int

@app.post("/api/settings")
def update_settings(payload: SettingsRequest):
    if payload.faction not in ("USEC", "BEAR"):
        raise HTTPException(status_code=400, detail="Фракция должна быть USEC или BEAR")
    if not (1 <= payload.level <= 79):
        raise HTTPException(status_code=400, detail="Уровень должен быть от 1 до 79")

    storage.set_faction(payload.faction)
    storage.set_level(payload.level)

```

```
return {"status": "ok", "faction": payload.faction, "level": payload.level}
```

Эндпоинт настроек. Ruydantic проверяет типы, мы дополнительно проверяем бизнес-смысл значений.

Запуск сервера:

```
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

- `app.main:app` — где искать приложение: модуль `app.main`, объект `app` внутри него
 - `--reload` — автоперезагрузка при изменениях
 - `--host 127.0.0.1` — слушать только локально (не из сети)
 - `--port 8000` — порт
-

`app/templates/base.html` — общий каркас

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}Tarkov Kappa Tracker{% endblock %}</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <header>
    <h1>Tarkov Kappa Tracker</h1>
    <nav>
      <a href="/">Главная</a>
      <a href="/tree">Дерево квестов</a>
    </nav>
  </header>

  <main>
    {% block content %}{% endblock %}
  </main>

  <footer>
    <small>Локальный трекер • Данные с tarkov.dev</small>
  </footer>
```

```
</body>
</html>
```

Базовый шаблон. Содержит общую структуру всех страниц: header, nav, footer.

`{% block title %}...{% endblock %}` — место, куда дочерние шаблоны вставляют своё. По умолчанию используется значение из `base`.

`{% block content %}{% endblock %}` — пустой блок, который дочерние шаблоны заполняют контентом страницы.

`<link rel="stylesheet" href="/static/style.css">` — подключаем CSS. Путь `/static/...` работает благодаря `app.mount("/static", ...)` в `main.py`.

`app/templates/index.html` — главная страница

```
{% extends "base.html" %}

{% block title %}Главная — Tarkov Kappa Tracker{% endblock %}

{% block content %}
<section class="stats">
    <h2>Прогресс до Кappa</h2>
    <p class="progress">
        Выполнено: <strong id="completed-count">{{ completed_count }}</strong>
        из <strong>{{ total_kappa }}</strong>
    </p>
    <p>Фракция: <strong>{{ faction }}</strong> • Уровень: <strong>{{ level }}</strong></p>
</section>

<section class="settings">
    <h2>Настройки персонажа</h2>
    <form id="settings-form">
        <label class="field">
            Фракция:
            <select name="faction" id="faction-select">
                <option value="USEC" {% if faction == "USEC" %}>selected{% endif %}
                <option value="BEAR" {% if faction == "BEAR" %}>selected{% endif %}
            </select>
        </label>
        <label class="field">
            Уровень:
            <input type="number" name="level" id="level-input"
                min="1" max="79" value="{{ level }}">
        </label>
    </form>
</section>
```

```

        </label>
        <button type="submit">Сохранить</button>
        <span id="settings-status" class="settings-status"></span>
    </form>
</section>

<section class="available">
    <h2>Доступно сейчас (<span id="available-count">{{ available|length }}</span>

    {% if available %}
    <ul class="quest-list" id="quest-list">
        {% for quest in available %}
        <li data-quest-id="{{ quest.id }}">
            <label>
                <input type="checkbox" class="quest-checkbox" data-quest-id="{{ q
                <span class="trader">[{{ quest.trader }}]</span>
                <span class="name">{{ quest.name }}</span>
                {% if quest.map %}
                <span class="map">— {{ quest.map }}</span>
                {% endif %}
            </label>
        </li>
        {% endfor %}
    </ul>
    {% else %}
    <p>Нет доступных квестов. Возможно, нужно повысить уровень.</p>
    {% endif %}
</section>

<script src="/static/app.js"></script>
{% endblock %}

```

Синтаксис Jinja2:

- {% extends "base.html" %} — наследование от базового шаблона
- {{ переменная }} — вывод значения
- {% if ... %}...{% endif %} — условие
- {% for x in ... %}...{% endfor %} — цикл
- {{ available|length }} — фильтр (длина списка)
- {% if faction == "USEC" %}selected{% endif %} — условный атрибут (выставляется выбранным то, что в прогрессе)

HTML-элементы:

- `<section>` — логический раздел страницы
 - `data-quest-id="..."` — кастомный data-атрибут для связи DOM с данными бэка
 - `<input type="checkbox">` — чекбокс
 - `<label>` оборачивает чекбокс — клик по тексту тоже срабатывает
 - `<form id="settings-form">` — форма для отправки данных
 - `<select>` и `<option>` — выпадающий список
 - `<input type="number" min="1" max="79">` — поле для числа с ограничениями
-

`app/templates/tree.html` — страница с графом

```
{% extends "base.html" %}
{% block title %}Дерево квестов — Tarkov Kappa Tracker{% endblock %}

{% block content %}
<section class="tree-section">
  <h2>Дерево квестов до Кappa</h2>
  <div class="legend">
    <span class="legend-item"><span class="dot completed"></span> Выполнено</span>
    <span class="legend-item"><span class="dot available"></span> Доступно</span>
    <span class="legend-item"><span class="dot locked"></span> Заблокировано</span>
  </div>
  <p class="hint">
    Колёсико — зум, перетаскивание — панорамирование.
    Клик на квест — подсветить цепочку предшественников.
  </p>
  <div id="cy"></div>
  <div id="quest-info" class="quest-info hidden">
    <h3 id="info-name"></h3>
    <p>Топровец: <strong id="info-trader"></strong></p>
    <p>Статус: <strong id="info-status"></strong></p>
    <p>Предшественников: <strong id="info-predecessors"></strong></p>
  </div>
</section>

<script src="https://cdnjs.cloudflare.com/ajax/libs/cytoscape/3.28.1/cytoscape.mi
<script src="https://cdnjs.cloudflare.com/ajax/libs/dagre/0.8.5/dagre.min.js"></s
<script src="https://cdn.jsdelivr.net/npm/cytoscape-dagre@2.5.0/cytoscape-dagre.m
<script src="/static/tree.js"></script>
```

```
{% endblock %}
```

`<div id="cy"></div>` — контейнер, в который Cytoscape будет рисовать граф.

Три скрипта подключают Cytoscape с CDN — публичных серверов, раздающих библиотеки. Для прототипов это удобно: не нужно ничего устанавливать, библиотека загрузится с интернета.

static/style.css — СТИЛИ

CSS — язык описания внешнего вида. Базовая идея: **селектор → набор свойств**.

```
body {  
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;  
    background: #1a1a1a;  
    color: #e0e0e0;  
}
```

`body` — селектор (выбирает тег). Внутри `{ }` — набор пар свойство:значение.

`#1a1a1a` — цвет в HEX. Первые две цифры — красный, потом зелёный, потом синий.

`#1a1a1a` — тёмно-серый.

```
.quest-list li:hover {  
    background: #333;  
}
```

`.quest-list` — селектор по классу (точка в начале). `li` — тег внутри элемента.

`:hover` — псевдокласс, срабатывает при наведении мышки.

```
.dot {  
    display: inline-block;  
    width: 12px;  
    height: 12px;  
    border-radius: 50%;  
}
```

`border-radius: 50%` делает квадрат кругом.

Полезно знать:

- Документация CSS: <https://developer.mozilla.org/ru/docs/Web/CSS>

- Игра для изучения селекторов: <https://flukeout.github.io/>

static/app.js — скрипт главной страницы

```
document.addEventListener("DOMContentLoaded", () => {
  const checkboxes = document.querySelectorAll(".quest-checkbox");

  checkboxes.forEach((checkbox) => {
    checkbox.addEventListener("change", async (event) => {
      const questId = event.target.dataset.questId;
      checkbox.disabled = true;

      try {
        const response = await fetch("/api/progress/toggle", {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ quest_id: questId }),
        });

        if (!response.ok) {
          throw new Error(`Сервер ответил ${response.status}`);
        }

        const data = await response.json();
        console.log("Сервер ответил:", data);

        document.getElementById("completed-count").textContent = data.com
        window.location.reload();

      } catch (error) {
        console.error("Ошибка:", error);
        alert("Не удалось обновить прогресс. Подробности в консоли.");
        checkbox.checked = !checkbox.checked;
        checkbox.disabled = false;
      }
    });
  });

  // Обработчик формы настроек
  const form = document.getElementById("settings-form");
  if (form) {
    form.addEventListener("submit", async (event) => {
      event.preventDefault();

      const faction = document.getElementById("faction-select").value;
```

```

const level = parseInt(document.getElementById("level-input").value,
const status = document.getElementById("settings-status");

try {
  const response = await fetch("/api/settings", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ faction, level }),
  });

  if (!response.ok) {
    const err = await response.json();
    throw new Error(err.detail || "Ошибка сервера");
  }

  status.textContent = "Сохранено ✓";
  status.className = "settings-status success";
  setTimeout(() => window.location.reload(), 500);

} catch (error) {
  console.error(error);
  status.textContent = "Ошибка: " + error.message;
  status.className = "settings-status error";
}

});
}
});

```

Разбор по частям:

`document.addEventListener("DOMContentLoaded", () => {...})` — «когда HTML полностью загрузился, выполни эту функцию». Без этого код мог бы запуститься раньше, чем появятся элементы.

`document.querySelectorAll(".quest-checkbox")` — найти все элементы с CSS-классом. Возвращает список (NodeList).

`.forEach((cb) => {...})` — пройти по каждому.

`addEventListener("change", async (event) => {...})` — повесить обработчик. `async` — функция асинхронная (внутри будут `await`).

`event.target.dataset.questId` — получить значение атрибута `data-quest-id`. В HTML атрибут с дефисом, в JS — `camelCase`. Это автоматическое преобразование.

`fetch(...)` — встроенная функция отправки HTTP-запросов. Аналог `httpx` в Python.

Возвращает Promise — асинхронное обещание результата.

`await` — ждать выполнения Promise и получить результат. Можно использовать только внутри `async`-функций.

`JSON.stringify({quest_id: questId})` — превратить JS-объект в JSON-строку.

`{ faction, level }` — сокращённая запись для `{ faction: faction, level: level }`.

Когда имя ключа совпадает с именем переменной.

`event.preventDefault()` — отменить стандартное поведение формы (перезагрузку страницы).

`parseInt(value, 10)` — превратить строку в число. 10 — основание системы счисления.

`try/catch` — обработка ошибок. Если внутри `try` что-то упадёт — управление перейдёт в `catch`.

`window.location.reload()` — перезагрузить страницу.

`setTimeout(callback, ms)` — выполнить `callback` через `ms` миллисекунд.

Полезные ссылки:

- Async/await: <https://learn.javascript.ru/async-await>
- Fetch API: <https://learn.javascript.ru/fetch>
- DOM (работа с элементами страницы): <https://learn.javascript.ru/document>

`static/tree.js` — отрисовка графа

Подробный разбор уже был в шаге 7. Ключевые моменты:

```
const cy = cytoscape({
  container: document.getElementById("cy"),
  elements: [...data.nodes, ...data.edges],
  style: [...],
  layout: { name: "dagre", rankDir: "TB" },
});
```

`cytoscape({...})` — создаёт граф. `[...data.nodes, ...data.edges]` — `spread`-оператор разворачивает оба массива в один.

```
cy.on("tap", "node", (event) => {  
    const node = event.target;  
    cy.elements().removeClass("highlighted");  
    const predecessors = node.predecessors();  
    node.addClass("highlighted");  
    predecessors.addClass("highlighted");  
});
```

`cy.on("tap", "node", ...)` — обработчик клика по узлу. `node.predecessors()` — встроенный метод обхода графа. `addClass("highlighted")` — добавить CSS-класс для подсветки.

Базовые концепции, которые ты освоил

Программирование на Python

- **Виртуальные окружения** — изоляция библиотек проекта
- **Импорты** — `import x`, `from x import y`, `from x import y as z`
- **Функции с типами** — `def fn(x: int) -> str:`
- **Списковые/множественные включения** — `[x for x in ...]`, `{x for x in ...}`
- **Лямбда-функции** — `lambda q: (q["trader"], q["name"])`
- **Контекстные менеджеры** — `with open(...) as f:` для гарантии закрытия ресурсов
- **Декораторы** — `@app.get(...)` навешивают функциональность
- **Идиома** `if __name__ == "__main__":` — отличает запуск от импорта

Работа с данными

- **JSON** — формат обмена данными между разными языками
- **Словари vs множества** — когда что использовать
- **Защита от None** — `obj.get("key")` или `obj["key"] if obj else default`
- **Идемпотентность** — операция, безопасная при повторном вызове

Графы

- **DAG** — направленный ациклический граф
- **Узлы и рёбра** — базовые элементы графа
- **Топологическая сортировка** — порядок без нарушения зависимостей
- **Транзитивный обход** — `nx.ancestors` , `nx.descendants`
- **Подграф** — часть графа по списку узлов

Web

- **HTTP** — протокол общения браузера с сервером
- **GET vs POST** — чтение vs изменение
- **REST API** — стиль построения API
- **GraphQL** — альтернатива REST
- **Статус-коды** — 200 (ok), 404 (не найдено), 422 (неверные данные), 500 (ошибка сервера)
- **JSON в теле запросов и ответов** — стандартный формат обмена
- **CORS, headers, content-type** — детали HTTP

FastAPI

- **Декораторы маршрутов** — `@app.get` , `@app.post`
- **Dependency Injection** — FastAPI сам подставляет зависимости по типам аргументов
- **Pydantic-модели** — описание структур и автовалидация
- **HTTPException** — стандартный способ возврата ошибок
- **Шаблоны Jinja2** — рендеринг HTML с подстановками
- **Статические файлы** — раздача CSS/JS

Frontend

- **HTML** — структура страницы
- **CSS** — внешний вид
- **JavaScript** — интерактивность

- **DOM** — представление страницы как дерева объектов
- **Event listeners** — реакция на клики, изменения
- **fetch + async/await** — асинхронные запросы из браузера
- **JSON.stringify / JSON.parse** — конвертация объектов и строк
- **Шаблонизация на сервере vs SPA** — наш подход vs реактивные приложения

Инструменты

- **VS Code** — редактор с расширениями для Python
- **PowerShell** — командная оболочка
- **DevTools (F12)** — отладка фронта
- `/docs` в **FastAPI** — автодокументация API
- **Логи uvicorn** — где смотреть, что приходит на сервер

Что делать, когда что-то идёт не так

Самый ценный навык программиста — **диагностика проблем**. Вот шпаргалка по всем ошибкам, с которыми мы столкнулись:

Python-ошибки

Ошибка	Что значит	Что делать
<code>ModuleNotFoundError: No module named 'X'</code>	Не найдена библиотека	Активировать <code>venv</code> , <code>pip install X</code>
<code>ModuleNotFoundError: No module named 'app'</code>	Запуск не из корня проекта	<code>cd C:\путь\к\проекту</code>
<code>IndentationError: unexpected indent</code>	Неправильные отступы	Проверить пробелы в начале строк
<code>NameError: name '__main__' is not defined</code>	Опечатка в <code>__name__</code>	Должно быть с двойными подчёркиваниями
<code>FileNotFoundError</code>	Не найден файл	Проверить путь, существует ли

		файл
<code>KeyError: 'X'</code>	Нет такого ключа в словаре	Использовать <code>.get("X")</code> или проверять заранее
<code>TypeError: unhashable type: 'dict'</code>	Передан словарь там, где нельзя	Проверить порядок аргументов (как с <code>TemplateResponse</code>)

HTTP-ошибки в браузере

Код	Что значит	Где искать причину
404 Not Found	URL не существует	Проверить декоратор <code>@app.post / @app.get</code> и URL
422 Unprocessable Entity	Pydantic не разобрал данные	Проверить структуру JSON в теле запроса
500 Internal Server Error	Ошибка в коде сервера	Смотреть traceback в терминале <code>uvicorn</code>
<code>Address already in use</code>	Порт занят	<code>--port 8001</code> или убить старый процесс

Где смотреть ошибки

1. **Браузер DevTools (F12)** → Console — JS-ошибки
2. **Браузер DevTools (F12)** → Network — что и куда отправлено, что вернулось
3. **Терминал uvicorn** — Python-ошибки с traceback
4. **Файлы JSON руками** — если данные подозрительные

Универсальный алгоритм отладки

1. Воспроизведи ошибку
2. Прочитай **последнюю строку** traceback — там суть
3. Найди в traceback **свой** файл (не библиотечный) и строку
4. Подумай, что в этой строке может быть не так
5. Если не понятно — добавь `print(...)` рядом и посмотри значения переменных

Как развивать проект дальше

Идеи для следующих шагов, в порядке сложности:

Простые улучшения

- Кнопка «обновить данные с tarkov.dev» прямо на сайте
- Группировка доступных квестов по торговцам
- Цвета фракций (USEC синий, BEAR красный)
- Подробная страница квеста с описанием задач (objectives)
- Поиск по квестам (input + фильтрация)

Средние улучшения

- Страница `/quests` со всеми Карра-квестами и фильтрами
- Привязка к определённому квесту-цели (не Карра, а например квест Collector от конкретного торговца)
- Подсказки «осталось до Карра: N квестов» в заголовке
- Сохранение нескольких профилей (если несколько персонажей)

Серьёзные улучшения

- Использовать SQLite вместо JSON для прогресса
- Авторизация (если делать для нескольких пользователей)
- Деплой на хостинг (Railway, Fly.io, VPS)
- Прогрессив веб-приложение (PWA) — установка на телефон как приложения
- Telegram-бот, использующий те же данные

Учебные задачи (для прокачки навыков)

- Покрыть код тестами (pytest)
- Настроить линтер (ruff, black) для автоформатирования
- Завести Git-репозиторий, выложить на GitHub
- Написать README.md с инструкцией запуска

- Сделать CLI-версию через click или typer
-

Важные ссылки для дальнейшего изучения

Python

- Официальный туториал: <https://docs.python.org/3/tutorial/>
- Учебник на русском: <https://pythontutor.ru/>
- Книга «Python Crash Course» — рекомендую для основ

Web

- HTML/CSS/JS на русском: <https://learn.javascript.ru/> + <https://developer.mozilla.org/ru/>
- FastAPI на русском: <https://fastapi.tiangolo.com/ru/>
- HTTP-протокол: <https://developer.mozilla.org/ru/docs/Web/HTTP>

Графы

- networkx tutorial: <https://networkx.org/documentation/stable/tutorial.html>
- Книга по алгоритмам — Кормен, «Алгоритмы: построение и анализ»

Инструменты

- VS Code: <https://code.visualstudio.com/docs>
 - Git (тебе пригодится): <https://git-scm.com/book/ru/v2>
-

Финальные мысли

За время этого проекта ты:

- Поднял Python-окружение с нуля
- Научился работать с HTTP-API
- Реализовал нетривиальную логику (граф зависимостей, топологическая сортировка)
- Поднял веб-сервер с шаблонами и интерактивностью

- Освоил отладку: чтение traceback, использование DevTools, диагностика по логам

Это **полный цикл** маленького веб-приложения — от данных до интерфейса. Все большие проекты строятся из таких же кирпичиков, просто их больше и они сложнее.

Главный принцип, который стоит унести: код пишется не сразу, а итерациями.

Сначала простейший рабочий вариант — потом улучшения. Сначала «работает на минимуме», потом «работает красиво». Никто не пишет идеально с первого раза — все пишут плохо, потом исправляют.

Пользуйся трекером, играй в Тарков, дорабатывай по мере необходимости. Это твой первый рабочий проект — он останется в портфолио и в памяти.

Удачи в Таркове и в коде! 🎮💻